Is Selenium Suitable for API Testing & Microservice Architectures?

The complete answer — with the right tools, code examples, and strategy

Python • Selenium • pytest • requests • Microservices | 2026

🎯 The Short Answer

Understanding what Selenium is and is not

❌ Selenium is NOT suitable for API testing.
Selenium controls a browser. APIs have no browser — no HTML, no buttons, no pages.
Selenium cannot send HTTP requests or parse JSON responses directly.

Selenium was built for one purpose — to automate a web browser. It clicks buttons, fills forms, reads text from pages, and navigates URLs. APIs operate at a completely different layer of the stack — they speak HTTP, return JSON or XML, and have no visual interface at all.

Aspect	UI Testing (Selenium)	API Testing (Different Tools)
What it controls	A real Chrome/Firefox browser	HTTP requests — no browser
How it interacts	Click, type, navigate pages	POST, GET, PUT, DELETE requests
Response format	HTML rendered on screen	JSON or XML data
Speed per test	2 – 30 seconds	10 – 200 milliseconds
Setup needed	Browser + WebDriver	Just Python + requests library
What it tests	Visual UI and user flows	Data, logic, and business rules

The Right Tools for API Testing

Python-based options that match your existing stack

Tier 1 — Python Tools (matches your current stack!)

Tool	Best For	Install Command
pytest + requests	REST API testing in Python — most popular	pip install requests
pytest + httpx	Async APIs, modern REST clients	pip install httpx
Tavern	YAML-based API test suites	pip install tavern

Tier 2 — Dedicated API Tools

Tool	Best For	Free?
Postman	Manual + automated API testing with GUI	Yes — free tier
Insomnia	Lightweight REST client	Yes — fully free
REST Assured	Java-based API testing	Yes — open source
K6	API performance and load testing	Yes — open source

Tier 3 — Microservice Specific

Tool	Best For	Free?
pytest + requests	Service-to-service integration testing	Yes — open source
Pact	Contract testing between microservices	Yes — open source
Locust	Load testing microservices at scale	Yes — open source
WireMock	Mock services for isolated testing	Yes — open source

What API Testing Looks Like in Python

Using pytest + requests on reqres.in (free practice API)

Here is a complete working example of a Python API test suite. This uses reqres.in — a free public API built for practice. The syntax is almost identical to your existing pytest Selenium tests.

```
# tests/api/test_user_api.py
import requests
```

```python
import pytest

BASE_URL = 'https://reqres.in/api'   # free practice API

class TestUserAPI:

    @pytest.mark.smoke
    def test_get_users_returns_200(self):
        """GET /users should return 200 OK."""
        response = requests.get(f'{BASE_URL}/users')
        assert response.status_code == 200

    @pytest.mark.smoke
    def test_get_users_returns_list(self):
        """Response body should contain a data array."""
        response = requests.get(f'{BASE_URL}/users')
        body = response.json()
        assert 'data' in body
        assert len(body['data']) > 0

    @pytest.mark.regression
    def test_create_user_returns_201(self):
        """POST /users should create user and return 201."""
        payload  = {'name': 'Sooraj', 'job': 'QA Engineer'}
        response = requests.post(f'{BASE_URL}/users', json=payload)
        assert response.status_code == 201
        body = response.json()
        assert body['name'] == 'Sooraj'
        assert 'id' in body

    @pytest.mark.regression
    def test_update_user_returns_200(self):
        """PUT /users/2 should update and return 200."""
        payload  = {'name': 'Sooraj Updated', 'job': 'Senior QA'}
        response = requests.put(f'{BASE_URL}/users/2', json=payload)
        assert response.status_code == 200
        assert 'updatedAt' in response.json()

    @pytest.mark.regression
    def test_delete_user_returns_204(self):
        """DELETE /users/2 should return 204 No Content."""
        response = requests.delete(f'{BASE_URL}/users/2')
        assert response.status_code == 204

    @pytest.mark.regression
    def test_invalid_user_returns_404(self):
        """GET /users/999 should return 404 Not Found."""
        response = requests.get(f'{BASE_URL}/users/999')
        assert response.status_code == 404
```

💡 Notice how similar this is to your Selenium pytest tests!
Same pytest markers (smoke, regression), same class structure,
same assert style — just requests instead of page objects.

Microservice Architecture — Full Testing Strategy

Where each tool belongs in the stack

In a microservice architecture, each service is tested independently at the API level. Selenium tests only the frontend — the single visual layer that sits on top of all the services combined.

Architecture Diagram

```
Microservice Architecture:

  Browser (User)
       |
       v
  [ Frontend UI ]  <-- Selenium tests THIS layer only
       |
  _____|_____________________________________
  |            |                 |              |
  v            v                 v              v
[ User     [ Order          [ Payment     [ Inventory
  Service]    Service]        Service]      Service]
  :8001       :8002           :8003          :8004

  ^            ^                 ^              ^
  |            |                 |              |
  pytest+requests tests EACH service independently
```

What Each Layer Tests

Layer	Tool	Tests	Speed
Frontend UI	Selenium	User flows, visual output, navigation	Slow — 2-30s/test
API Gateway	pytest+requests	Routes, auth, rate limits	Fast — <1s/test
User Service	pytest+requests	Register, login, profile CRUD	Fast — <1s/test
Order Service	pytest+requests	Create order, update, cancel	Fast — <1s/test
Payment Service	pytest+requests	Charge, refund, validation	Fast — <1s/test
Contracts	Pact	Service A talks correctly to Service B	Medium
Load	Locust	Services handle 1000 concurrent users	Variable

Combined Project Structure

UI tests + API tests in one pytest suite

The best approach for an enterprise project is to combine both UI and API tests in the same repository, sharing the same pytest configuration, markers, and HTML report.

```
full_test_suite/
|
+-- ui_tests/                    <- Selenium (what you built!)
|   +-- pages/
|   |   +-- base_page.py
|   |   +-- login_page.py
|   |   +-- inventory_page.py
|   +-- tests/
|   |   +-- conftest.py
|   |   +-- test_login.py
|   |   +-- test_checkout.py
|   +-- utils/
|
+-- api_tests/                   <- pytest + requests (new!)
|   +-- utils/
|   |   +-- api_client.py        # base HTTP client class
|   |   +-- test_data.py         # shared dynamic data
|   +-- tests/
|   |   +-- test_user_api.py
|   |   +-- test_order_api.py
|   |   +-- test_payment_api.py
|   +-- conftest.py
|
+-- contract_tests/              <- Pact (microservices)
|   +-- test_contracts.py
|
+-- config/
|   +-- config.py               # shared URLs and credentials
|
+-- pytest.ini                  # runs ALL layers together!
+-- requirements.txt
+-- .env                        # secrets - never commit!
+-- .gitignore
```

Run Commands for Combined Suite

```
# Run ALL tests (UI + API together)
pytest -n 3 -v

# Run only API tests
pytest api_tests/ -v

# Run only UI tests
pytest ui_tests/ -v

# Run smoke tests across both layers
pytest -m smoke -n 3 -v

# Run regression nightly
pytest -m regression -n 5 -v
```

The Testing Pyramid

The industry-standard strategy for test coverage

The Testing Pyramid is the industry-standard guide to how many tests of each type to write. The key rule: the faster and cheaper the test, the more of them you should have.

```
                /\
               /  \
              / E2E \     <- Selenium UI (10%)
             /  Tests\      Slow, expensive, few
            /————————\    ~10-20 tests
           /   API      \
          /    Tests     \ <- pytest+requests (20%)
         /————————————————\  Medium speed, reliable
        /   Unit  Tests    \  ~100-200 tests
       /———————————————————\ <- pure pytest (70%)
      /                     \ Fast, cheap, many
     /———————————————————————\ ~500+ tests
```

Layer	Type	Coverage	Speed	Count
Top	E2E / UI (Selenium)	Full user flows	2-30s each	10% — fewest
Middle	API / Integration	Endpoints + data	<1s each	20% — some
Bottom	Unit Tests	Functions + logic	<0.1s each	70% — most

💡 Key insight: Don't test everything through the UI.
If a business rule can be tested at the API level in 50ms,
don't spend 10 seconds testing it through Selenium.
Use Selenium only for what only Selenium can test — the visual user experience.

Free Practice APIs

Public APIs built for learning and practice

These are free, publicly accessible APIs — no signup needed for most. Perfect for building your first API test suite.

API	URL	Good For
ReqRes	https://reqres.in	Users CRUD — perfect for learning REST
JSONPlaceholder	https://jsonplaceholder.typicode.com	Posts, comments, todos — simple
restful-booker	https://restful-booker.herokuapp.com	Hotel booking — auth + full CRUD

PetStore	https://petstore.swagger.io	Swagger docs + complex API flows
Fake Store API	https://fakestoreapi.com	E-commerce — products, cart, users
Open Notify	http://api.open-notify.org	ISS position — simple GET requests

✅ Summary — The Complete Answer

One page reference for everything covered

Question	Answer
Selenium for API testing?	No — wrong tool. Selenium controls browsers only.
Best Python API tool?	pytest + requests — familiar syntax, same pytest suite
Selenium role in microservices?	Frontend UI layer only — not individual services
Can UI + API tests coexist?	Yes — same repo, same pytest.ini, shared report
Free API to practice on?	reqres.in or restful-booker.herokuapp.com
Testing pyramid split?	70% unit / 20% API / 10% UI (Selenium)
Microservice contract testing?	Use Pact — verifies services talk to each other correctly
Load testing microservices?	Use Locust — simulates thousands of concurrent users

✅ Next step: Build an API test suite using pytest + requests on restful-booker.herokuapp.com — same step-by-step approach as the saucedemo Selenium suite, running in the same project!

Selenium vs API Testing Guide | 2026